Python Short-Term Trading Backtesters

https://gemini.google.com/app/622d63acf9254892

User prompt: Can i code a short term trading strategy tester in python for for example sessions or other popular strategies. What strategies could be coded? Prioritize with functionality with python

Response: Yes, Python is exceptionally well-suited for building short-term trading strategy testers (backtesters) due to its powerful data science and financial libraries.

Essential Python Libraries for Backtesting

- **Pandas & NumPy:** The core foundations for handling time-series price data, calculating indicators, and running vectorized calculations.
- **VectorBT:** Highly optimized for speed, allowing you to test thousands of parameter combinations simultaneously using NumPy arrays.
- **Backtrader:** A feature-rich framework specifically designed for event-driven backtesting, multi-asset data, and plotting trades.
- **yfinance or CCXT:** APIs used to pull historical stock, forex, or crypto data (CCXT is ideal for short-term crypto session data).

Short-Term Strategies to Code (Prioritized by Python Functionality)

1. Moving Average Crossover (Trend Following)

- **How it works:** Generates a buy signal when a fast moving average (e.g., 9-period EMA) crosses above a slow moving average (e.g., 21-period EMA), and a sell signal when it crosses below.
- **Python Implementation:** Extremely simple using `pandas` rolling windows (`df['close'].rolling(window=9).mean()`).

2. Session Open Breakout (Time-Based)

- **How it works:** Records the high and low price of a specific market session's opening range (like the first 15 or 30 minutes of the London or New York stock/forex session) and trades breakouts beyond those boundaries.
- **Python Implementation:** Requires filtering datetime indices (`df.index.hour == 8` and `df.index.minute < 30`) to isolate the session window, then tracking subsequent highs and lows.

3. RSI Mean Reversion

- **How it works:** Uses the Relative Strength Index (RSI) to identify overbought (>70) or oversold (<30) conditions over short timeframes (e.g., 5-minute or 15-minute charts), betting on a price bounce back to the mean.
- **Python Implementation:** Can be calculated easily using native pandas logic or specialized packages like `ta` (Technical Analysis library for Python).

4. Bollinger Band Squeeze & Breakout

- **How it works:** Monitors periods of low volatility where Bollinger Bands contract (squeeze), anticipating a sharp breakout movement in either direction when volatility expands.
- **Python Implementation:** Involves calculating rolling standard deviations and moving averages to plot upper and lower bands, then triggering trades when price breaches the outer bands.

User prompt: Could you write me a clear description of the moving average crossover which i can paste into my coding AI. Describe it as specifically as possible and tell what to code, how to code a clean interactive dashboard with all important info such as comparing the strategy to sp500 buy hold, profit ratio and winrate (€ and RR). Please provide a downloadable text file rather than a chat answer

Response: Your downloadable prompt specification text file is ready.

[file-tag: code-generated-file-8beab5cd-b28d-4d5c-8732-94fcc86e972e]

What Is Included in the Text File:

1. **Strategy Specification & Logic:**
    - Detailed Fast MA vs. Slow MA (e.g., 9 EMA vs. 21 EMA) crossover rules.
    - Execution timing rules designed to prevent lookahead bias (executing on the bar following the signal).
    - Configurable transaction costs and slippage.
2. **Benchmark Comparison:**
    - Direct comparison against an S&P 500 Buy & Hold benchmark (`SPY`) normalized over the same timeframe.
3. **Required Metrics & Analytics:**
    - Total Return (%), Alpha / Excess Return.
    - Win Rate (%) and Win/Loss currency values (€/$).
    - Profit Ratio / Profit Factor (Gross Gains / Gross Losses).
    - Realized Risk-to-Reward (R:R) Ratio per trade.
    - Max Drawdown (%) and Sharpe Ratio.
4. **Dashboard Layout & UI Requirements:**
    - Streamlit sidebar inputs for parameters (MA lengths, timeframe, initial capital, asset ticker).
    - Interactive Plotly equity curve chart comparing the strategy to the S&P 500.
    - Candlestick price chart with overlay MAs and visual Buy/Sell arrow markers.
    - Detailed downloadable CSV trade log.
5. **Code Architecture:**

- Modular structure divided into data ingestion (`yfinance`), indicator calculation, backtest execution engine, analytics calculation, and Plotly visualization functions.

User prompt: The code gave a problem like the picture attached could you help me fix it

Response: This warning happens because the script was executed using standard Python (`python ma_crossover_backtester.py`) instead of launching it through the Streamlit engine. Streamlit needs its own CLI runner to initialize the web dashboard context; otherwise, it operates in "bare mode" and prints a `missing ScriptRunContext!` warning for every Streamlit call.

How to Fix It

1. Open your terminal or command prompt in the directory where your script is saved.
2. Launch the script using the Streamlit execution command:

   ```
   streamlit run ma_crossover_backtester.py
   ```

3. If your browser does not open automatically, copy and paste the `Local URL` shown in your terminal (usually `http://localhost:8501`) into your web browser.

Additional Troubleshooting Tips

- **Ensure dependencies are installed:** If the command is not recognized, make sure you have installed the required packages in your active Python environment:

  ```
  pip install streamlit yfinance pandas numpy plotly
  ```

- **Virtual Environments:** If you are using VS Code or PyCharm, ensure your terminal is set to the correct Python interpreter/virtual environment where Streamlit is installed.

User prompt: Where i could run my code then

Response: 1. Locally on Your Computer (Terminal / VS Code)

- **Command Line:** Open Command Prompt, PowerShell, or macOS Terminal in the folder where your `ma_crossover_backtester.py` file is stored.
- **Execution:** Run `streamlit run ma_crossover_backtester.py`. Streamlit starts a local web server and automatically opens the dashboard in your browser at `http://localhost:8501`.
- **IDEs:** In editors like VS Code or PyCharm, open the built-in terminal, activate your Python virtual environment, and run the same `streamlit run` command.

2. Streamlit Community Cloud (Free Online Web Hosting)

- **GitHub Repository:** Push your Python code (`ma_crossover_backtester.py`) and a `requirements.txt` file (listing `streamlit`, `yfinance`, `pandas`, `numpy`, and `plotly`) to a free GitHub repository.

- **One-Click Deployment:** Connect your GitHub account to Streamlit Community Cloud and select your repository.
- **Shareable Link:** Streamlit builds and hosts the dashboard online for free, providing a shareable web link to access the app from any browser or phone.

3. Google Colab (Notebook with Tunneling)

- **Cloud Notebook:** Open a Google Colab notebook and install the required libraries with `!pip install streamlit yfinance pandas numpy plotly pyngrok`.
- **Execution & Tunneling:** Save your script in the Colab session using `%%writefile app.py`, start the server with `!streamlit run app.py &`, and use a tunneling tool like `ngrok` or `localtunnel` to create a public temporary URL to view the live dashboard.

4. Cloud Application Platforms

- **Free & Low-Cost Hosts:** Web platforms like **Render**, **Ploomber Cloud**, **Koyeb**, or **Google Cloud Run** can build and run Streamlit applications directly from GitHub or container builds.

Check out The Easiest Way to Deploy A Streamlit App to see step-by-step instructions on uploading your code to GitHub and hosting it online for free using Streamlit Community Cloud.

User prompt: i tried powershell and it gave this error even though the folder was the one where my ma_crossover_backtester.py lies. what now

Response: Run the command using Python's module flag (`-m`), which bypasses Windows PATH issues by telling Python directly to execute the Streamlit module:

```
python -m streamlit run ma_crossover_backtester.py
```

If you still encounter errors, follow these steps to resolve the environment issue:

1. Install Streamlit in Your Current Environment

If Streamlit was not previously installed in the Python environment PowerShell is targeting, run:

```
pip install streamlit yfinance pandas numpy plotly
```

2. Use the Python Launcher (`py`)

If `python` itself is not recognized by PowerShell, use the default Windows launcher:

```
py -m streamlit run ma_crossover_backtester.py
```

3. Restart PowerShell

If you recently installed Streamlit while PowerShell was already open, environment variables may not have refreshed. Close PowerShell, re-open it in your file's folder, and run the command again.
